

RAPPORT DE PROJET

Pipeline d'analyse des fuites de données publiques

Have I Been Pwned (HIBP)

Groupe	Groupe 4
Étudiant	TCHA-TCHESSI E. Aristide, PLANTE Richard, DJANGBIOGOU MONDO Joseph
Formation	Master 1
Dépôt GitHub	github.com/Aristo007/hibp-data-breaches-pipeline
Bucket AWS S3	hibp-groupe10-2026 (région eu-north-1)
Date du rapport	8 août 2026

Sommaire

1. Contexte et problématique
2. Objectifs du projet
3. Architecture du pipeline
4. Stack technique
5. Méthodologie et travail réalisé, semaine par semaine
6. Résultats et analyses
7. Le dashboard Power BI
8. Difficultés rencontrées et solutions apportées
9. Limites et esprit critique
10. Récapitulatif des livrables
11. Conclusion et perspectives

1. Contexte et problématique

Les fuites de données (data breaches) se multiplient chaque année et touchent tous les secteurs d'activité. Ce projet a pour objectif de construire un pipeline de données complet, permettant de collecter, transformer, stocker et visualiser les informations publiques relatives à ces incidents, afin d'en dégager des tendances exploitables : évolution dans le temps, volumes de comptes compromis, types de données les plus fréquemment volés, et répartition par secteur d'activité.

La source de données retenue est l'API publique Have I Been Pwned (HIBP), qui recense de manière structurée les principales fuites de données connues depuis 2007. L'endpoint utilisé (/breaches) est public et ne nécessite aucune authentification, ce qui a permis de développer le pipeline sans dépendance à un compte payant.

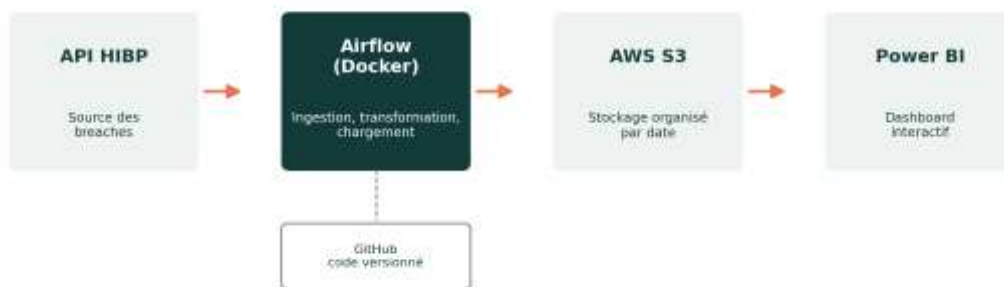
Question de départ retenue : « Comment construire un pipeline automatisé, reproductible et documenté pour transformer des données brutes de fuites en informations exploitables ? »

2. Objectifs du projet

- Construire un pipeline d'ingestion automatisé interrogeant l'API HIBP en Python
- Orchestrer ce pipeline avec Apache Airflow, exécuté dans des conteneurs Docker
- Stocker les données transformées sur AWS S3, organisées par date d'ingestion
- Produire un dashboard Power BI avec au minimum 5 visualisations et des filtres interactifs
- Documenter l'ensemble du projet sur un dépôt GitHub public, avec schéma d'architecture et instructions d'installation

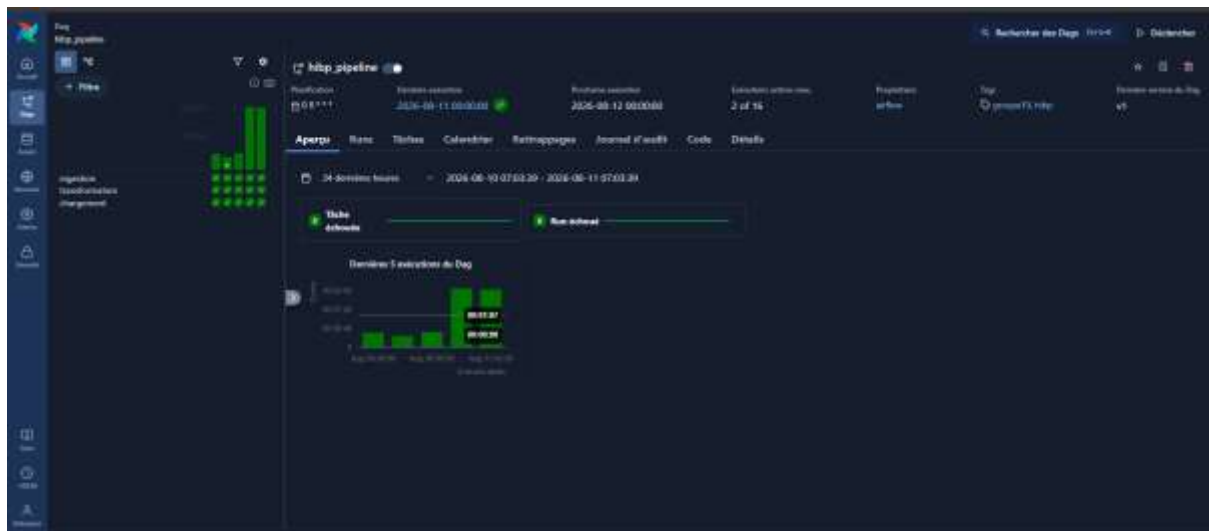
3. Architecture du pipeline

Le pipeline suit un flux linéaire en quatre grandes étapes, orchestrées par Airflow :



Le DAG Airflow (nommé `hibp_pipeline`) est composé de trois tâches séquentielles. La tâche d'ingestion télécharge la liste complète des breaches depuis l'API HIBP. La tâche de transformation génère deux fichiers CSV exploitables par Power BI (un pour les breaches, un pour les types de données volées). La tâche de chargement envoie ces fichiers vers le bucket S3, organisés selon le chemin `breaches/{date}/`, avec une gestion d'erreurs robuste : en l'absence d'identifiants AWS configurés, le pipeline bascule automatiquement en mode simulation plutôt que d'échouer, ce qui a permis de développer et tester le DAG de façon autonome avant l'obtention des accès AWS définitifs.

Le pipeline intègre également des mécanismes de résilience : nouvelles tentatives automatiques (retries, 2 essais) en cas d'échec réseau temporaire, délai maximal d'exécution par tâche (timeout de 5 minutes), et messages d'erreur explicites en cas de problème (API indisponible, fichier corrompu, identifiants invalides).



Vue d'ensemble du DAG dans Airflow

4. Stack technique

Outil	Rôle dans le projet
Python (requests, boto3)	Ingestion de l'API HIBP et envoi des fichiers vers S3
Apache Airflow 3.3.0	Orchestration du pipeline (3 tâches, retries, gestion d'erreurs)
Docker / Docker Compose	Exécution d'Airflow en environnement conteneurisé
AWS S3	Stockage des données transformées, organisées par date
Power BI Desktop	Construction du dashboard et des visualisations interactives
GitHub	Versionnement du code et documentation du projet

5. Méthodologie et travail réalisé

5.1 Ingestion et analyses en Python

- Exploration de l'API HIBP v3 : identification de l'endpoint public /breaches (1 020 breaches récupérés, aucune clé API requise)
- Script Python structuré en fonctions (téléchargement, sauvegarde JSON, analyses), commenté et versionné sur GitHub dès les premières étapes
- Identification du plus gros breach et calcul du volume cumulé de comptes compromis
- Analyse de l'évolution du nombre de breaches par année (2007–2026)
- Classement des types de données les plus fréquemment volés
- Classification manuelle progressive par secteur d'activité, avec une catégorie dédiée pour les compilations de données non attribuables à un site source unique (stealer logs, combolists)
- Export des données en fichiers CSV structurés (export_breaches.csv, export_dataclasses.csv) pour l'intégration dans Power BI

```
PS C:\Users\Lenovo\hibp-projet> python3 hibp_pipeline.py
[OK] 1026 breaches téléchargés depuis l'API.
[OK] Sauvegardé dans breaches.json
```

```
--- Plus gros breach ---
```

```
Nom : SynthientCredentialStuffingThreatData
```

```
Comptes touchés : 1,957,476,021
```

```
Date : 2025-04-11
```

```
Total cumulé (tous breaches) : 17,777,846,158
```

```
--- Nombre de breaches par année ---
```

```
2007 : 1 breaches
```

```
2008 : 3 breaches
```

```
2009 : 2 breaches
```

```
2010 : 4 breaches
```

```
2011 : 19 breaches
```

```
2012 : 20 breaches
```

```
2013 : 31 breaches
```

```
2014 : 49 breaches
```

```
2015 : 76 breaches
```

```
2016 : 104 breaches
```

```
2017 : 54 breaches
```

```
2018 : 78 breaches
```

```
2019 : 82 breaches
```

```
2020 : 98 breaches
```

```
2021 : 78 breaches
```

```
2022 : 61 breaches
```

```
2023 : 46 breaches
```

```
2024 : 85 breaches
```

```
2025 : 61 breaches
```

```
2026 : 74 breaches
```

```
--- Top 15 des types de données volées ---
```

```
Email addresses : 1019 breaches
```

```
Passwords : 673 breaches
```

```
Names : 546 breaches
```

```
Usernames : 481 breaches
```

```
IP addresses : 386 breaches
```

```
Phone numbers : 367 breaches
```

```
Physical addresses : 296 breaches
```

```
Dates of birth : 282 breaches
```

```
Genders : 194 breaches
```

```
Geographic locations : 145 breaches
```

```
Purchases : 84 breaches
```

```
Website activity : 77 breaches
```

```
Social media profiles : 50 breaches
```

```
Job titles : 48 breaches
```

```
Employers : 39 breaches
```

```

--- Top 20 domaines à classer par secteur (par PwnCount) ---

```

	1,957,476,021 comptes	SynthientCredentialStuffingThreatData
	772,904,991 comptes	Collection1
verifications.io	763,117,241 comptes	VerificationsIO
	711,477,622 comptes	OnlinerSpamBot
	622,161,052 comptes	PDL
	593,427,119 comptes	ExploitIn
facebook.com	509,458,528 comptes	Facebook
	457,962,538 comptes	AntiPublic
rivercitymediaonline.com	393,430,309 comptes	RiverCityMedia
	361,468,099 comptes	TelegramCombolists
myspace.com	359,420,698 comptes	MySpace
	284,132,969 comptes	AlienStealerLogs
	282,478,425 comptes	NotSOCRadar
wattpad.com	268,765,495 comptes	Wattpad
163.com	234,842,089 comptes	NetEase
deezer.com	229,037,936 comptes	Deezer
citoday.in	226,883,414 comptes	Citoday
twitter.com	211,524,284 comptes	Twitter200M
	182,962,095 comptes	SynthientStealerLogThreatData
zynga.com	172,869,660 comptes	Zynga

```

--- Volume de comptes compromis par secteur ---
Compilation / Non attribuable : 7,439,338,636 comptes
Non classé : 6,962,398,148 comptes
Marketing / Données email : 1,156,547,550 comptes
Réseaux sociaux : 1,087,163,230 comptes
Divertissement / Média : 268,765,495 comptes
Services email / Technologie : 234,842,089 comptes
Divertissement / Streaming : 229,037,936 comptes
Marché noir de données : 226,883,414 comptes
Jeux vidéo : 172,869,660 comptes
[OK] Export CSV créé : export_breaches.csv
[OK] Export CSV créé : export_dataclasses.csv
PS C:\Users\Lenovo\hibp-projet>

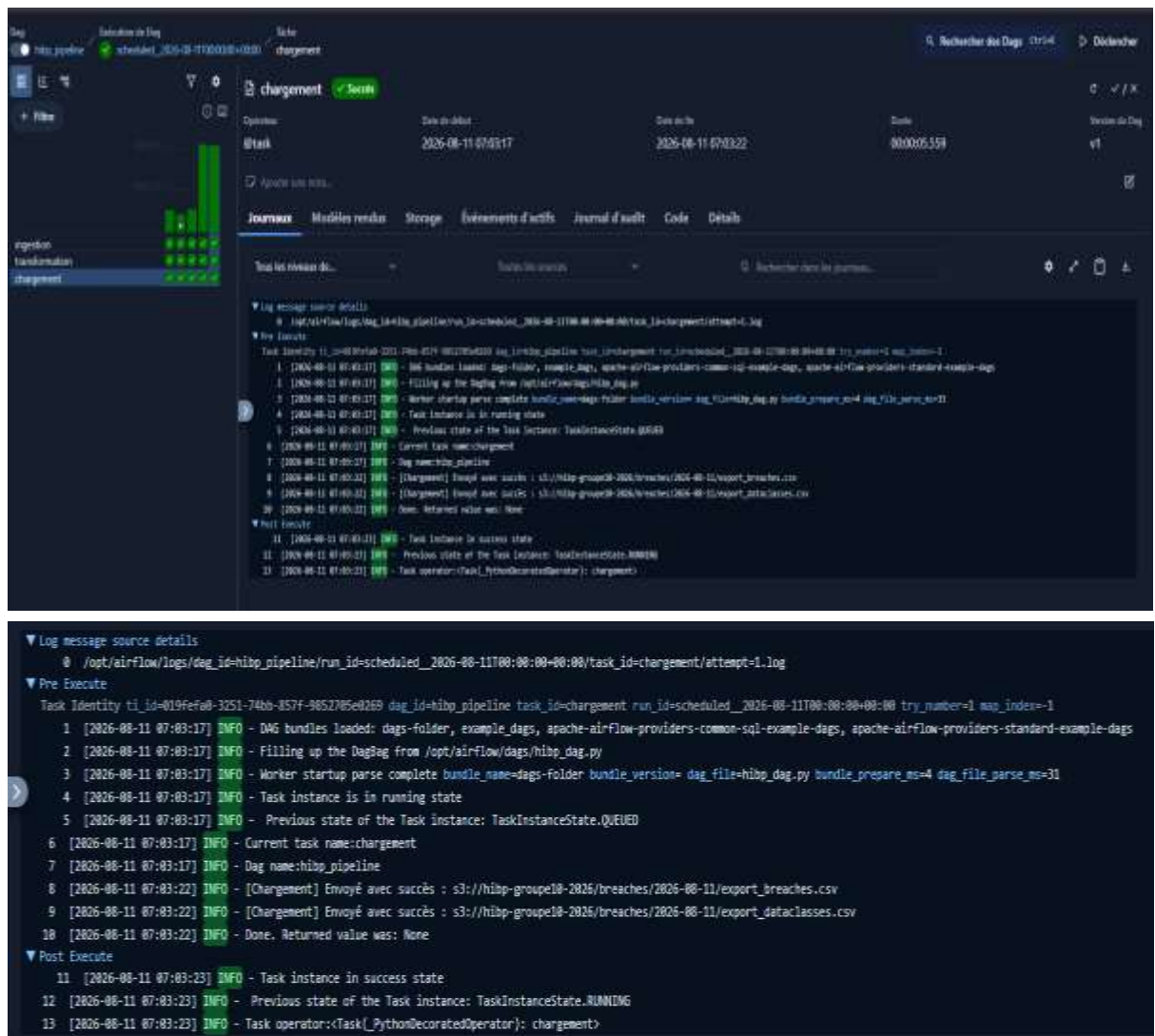
```

Exécution de « python3 hibp_pipeline.py », montrant les 4 analyses affichées (plus gros breach, évolution par année, types de données, secteurs).

5.2 Orchestration avec Docker et Apache Airflow

- Installation et configuration d'Apache Airflow via Docker Compose (fichier officiel Apache, 4 dossiers requis : dags, logs, plugins, config)
- Construction d'un DAG à 3 tâches avec l'API moderne d'Airflow (décorateurs @dag et @task)
- Ajout de mécanismes de robustesse : retries automatiques, timeout, gestion explicite des erreurs réseau et de fichiers corrompus
- Résolution de plusieurs incidents techniques d'infrastructure : pannes ponctuelles du moteur Docker Desktop (erreurs « read-only file system », conteneurs marqués « unhealthy »), diagnostiquées via les journaux (logs) des conteneurs et résolues par redémarrage complet de Docker Desktop et de WSL2

- Préparation du code d'envoi S3 (bibliothèque boto3) avec bascule automatique en mode simulation tant que les identifiants AWS n'étaient pas disponibles, permettant de ne pas bloquer le développement du DAG



The screenshot displays the Apache Airflow web interface for a task named 'chargement'. The task is in a 'Succès' (Success) state. The logs show the task's execution, including bundle loading, DAG filling, worker startup, and successful upload of data to S3.

```

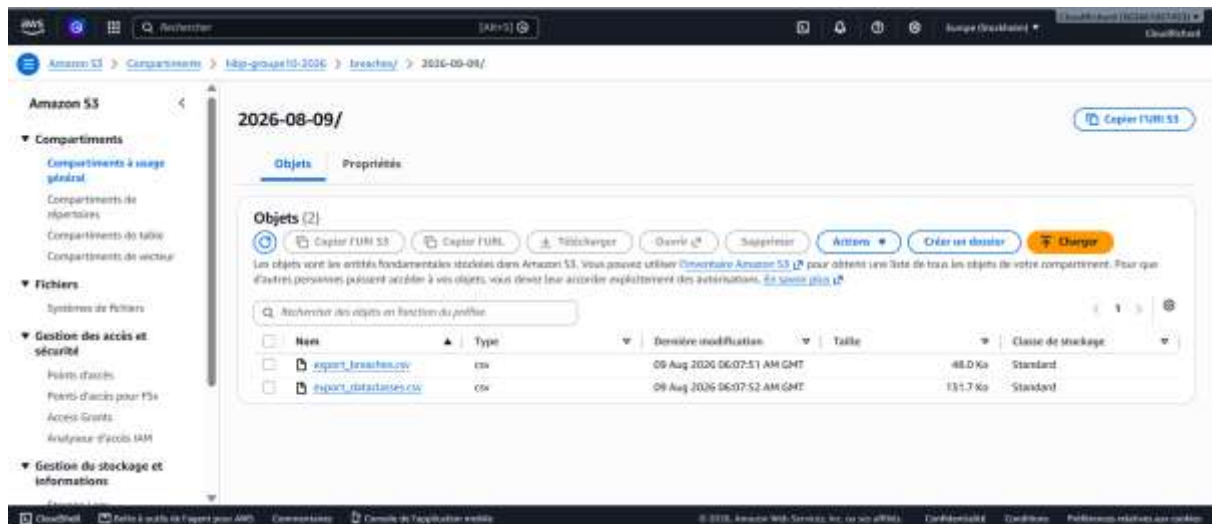
▼ Log message source details
0 /opt/airflow/logs/dag_id=hibp_pipeline/run_id=scheduled_2026-08-11T00:00:00-00:00/task_id=chargement/attempt-1.log
▼ Pre Execute
Task Identity ti_id=019fef08-3251-74bb-857f-9052705e0269 dag_id=hibp_pipeline task_id=chargement run_id=scheduled_2026-08-11T00:00:00-00:00 try_number=1 map_index=1
1 [2026-08-11 07:03:17] INFO - DAG bundles loaded: dags-folder, example_dags, apache-airflow-providers-common-sql-example-dags, apache-airflow-providers-standard-example-dags
2 [2026-08-11 07:03:17] INFO - Filling up the DagBag from /opt/airflow/dags/hibp_dag.py
3 [2026-08-11 07:03:17] INFO - Worker startup parse complete bundle_name=dags-folder bundle_version= dag_file=hibp_dag.py bundle_prepare_ms=4 dag_file_parse_ms=31
4 [2026-08-11 07:03:17] INFO - Task instance is in running state
5 [2026-08-11 07:03:17] INFO - Previous state of the Task Instance: TaskInstanceState.QUEUED
6 [2026-08-11 07:03:17] INFO - Current task name: chargement
7 [2026-08-11 07:03:17] INFO - Dag name: hibp_pipeline
8 [2026-08-11 07:03:22] INFO - [Chargement] Envoyé avec succès : s3://hibp-groupe10-2026/breaches/2026-08-11/export_breaches.csv
9 [2026-08-11 07:03:22] INFO - [Chargement] Envoyé avec succès : s3://hibp-groupe10-2026/breaches/2026-08-11/export_dataclasses.csv
10 [2026-08-11 07:03:22] INFO - Done. Returned value was: None
▼ Post Execute
11 [2026-08-11 07:03:23] INFO - Task instance in success state
12 [2026-08-11 07:03:23] INFO - Previous state of the Task Instance: TaskInstanceState.RUNNING
13 [2026-08-11 07:03:23] INFO - Task operator: <Task_PythonDecoratedOperator>: chargement>

```

Journaux de la tâche « chargement »

5.3 Stockage AWS S3

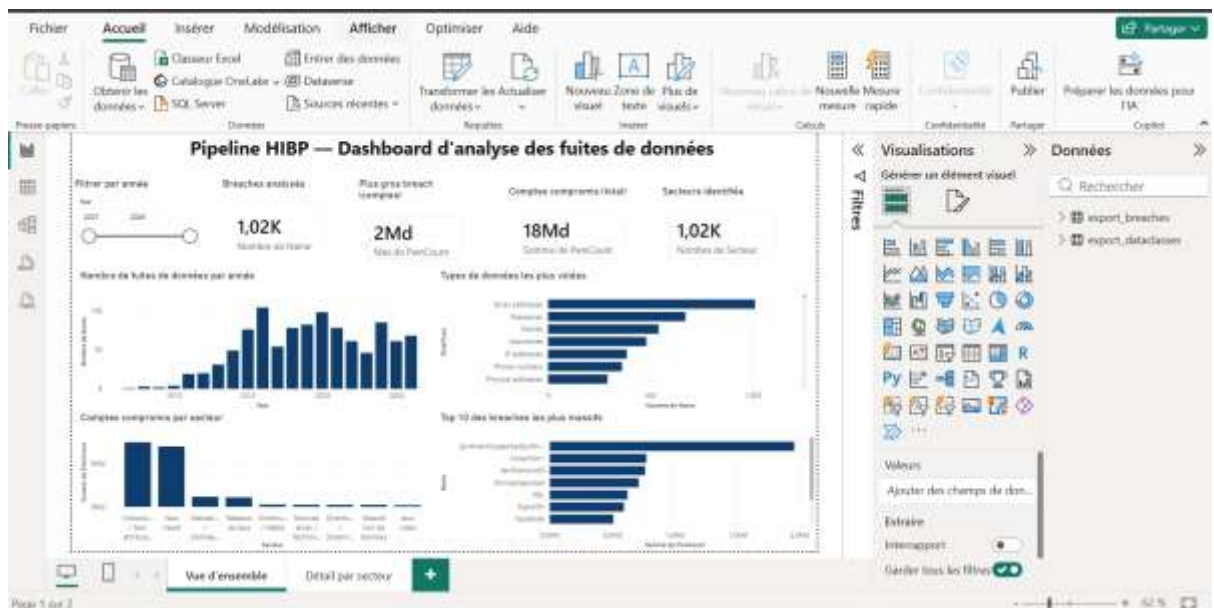
- Création d'un compte AWS et d'un bucket S3 dédié (hibp-groupe10-2026, région eu-north-1)
- Création d'un utilisateur IAM (hibp-projet-groupe10) avec permissions limitées à S3 (politique AmazonS3FullAccess), plutôt que l'utilisation d'identifiants racine du compte
- Connexion du pipeline Airflow au bucket réel via les variables d'environnement (.env, non versionné) : les fichiers sont envoyés et organisés par date d'ingestion (breaches/{date}/)
- Vérification manuelle de la présence des fichiers sur le bucket après exécution du DAG



Contenu du bucket S3 sur la console AWS

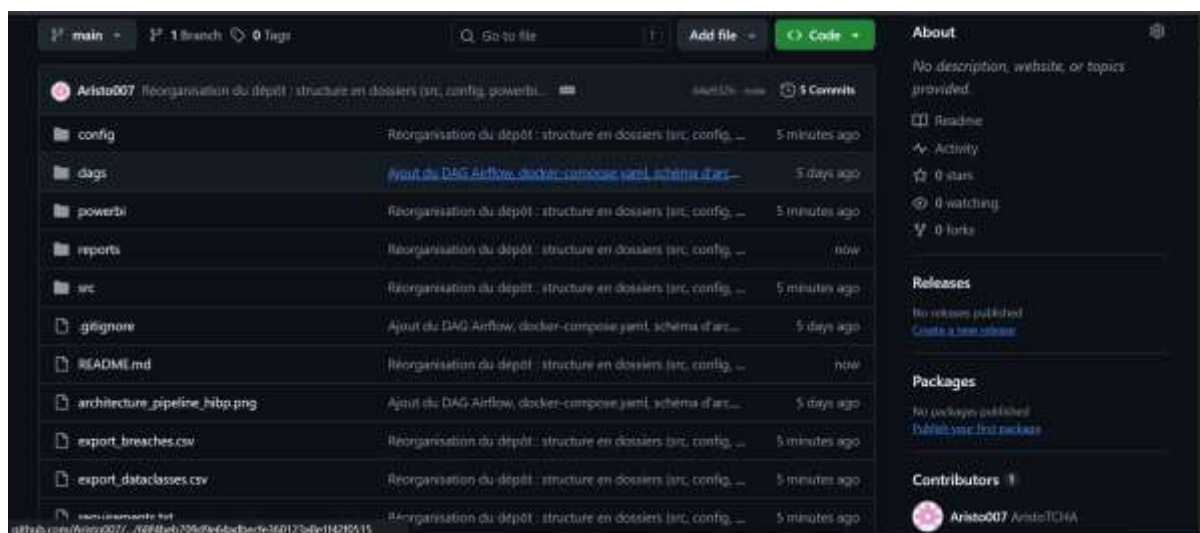
5.4 Dashboard Power BI

- Import des fichiers CSV (export_breaches.csv, export_dataclasses.csv) et modélisation des données : relation « plusieurs à un » entre les deux tables, sur le champ Name
- Construction de 5 visualisations : évolution des fuites par année, comptes compromis par secteur, types de données les plus volés, top 10 des breaches les plus massifs, et un filtre interactif par année
- Le filtre interactif (segment) pilote automatiquement l'ensemble des autres visuels de la page



5.5 Documentation et GitHub

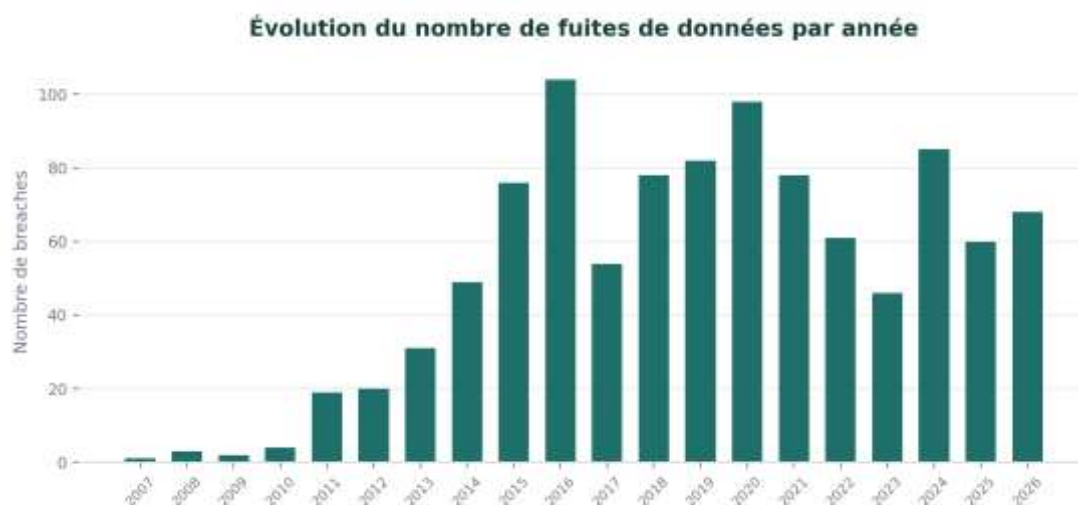
- Dépôt GitHub public, structuré et versionné au fil du projet (plusieurs commits successifs)
- README complet : description du projet, architecture, stack technique, instructions d'installation reproductibles pas à pas
- Schéma d'architecture illustrant le flux de données, intégré au README
- Sécurisation stricte du dépôt : exclusion systématique des identifiants AWS et données sensibles via .gitignore (.env, fichiers de clés IAM, données brutes JSON)



Page d'accueil du dépôt GitHub

6. Résultats et analyses

Indicateur	Valeur
Nombre total de breaches analysés	1 020
Plus gros breach	SynthientCredentialStuffingThreatData — $\approx$ 1,96 milliard de comptes (avril 2025)
Total cumulé de comptes compromis	$\approx$ 17,7 milliards (tous breaches confondus)
Type de donnée le plus volé	Adresses email — présentes dans 99 % des breaches
Deuxième type le plus volé	Mots de passe — présents dans 66 % des breaches
Année avec le plus de breaches recensés	2016 (plus de 100 breaches)

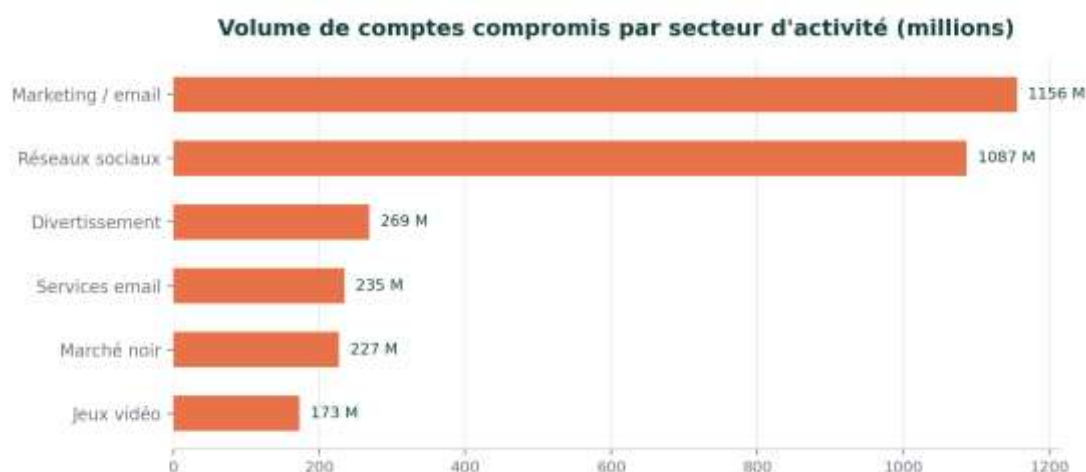


Le total cumulé de comptes compromis ($\approx$ 17,7 milliards) dépasse la population mondiale, ce qui s'explique par le fait qu'une même personne peut apparaître dans plusieurs breaches distincts si elle a réutilisé les mêmes

identifiants sur plusieurs services. Ce point a volontairement été assumé comme une limite de lecture des données, plutôt que dissimulé.

7. Le dashboard Power BI

Le dashboard final regroupe cinq visualisations couvrant les principaux axes d'analyse demandés : évolution temporelle, répartition par secteur, typologie des données volées, et hiérarchisation des incidents les plus massifs. Un filtre interactif par année permet d'explorer dynamiquement l'ensemble des visuels.



Ce graphique illustre une limite méthodologique assumée : les catégories « Non classé » et « Compilation / Non attribuable » (non représentées ici, car sans secteur identifiable) concentrent la majorité du volume total, faute d'un champ « secteur d'activité » fourni nativement par l'API HIBP. Les secteurs présentés ci-dessus correspondent uniquement aux domaines qu'il a été possible de rattacher manuellement à une catégorie.

8. Difficultés rencontrées et solutions apportées

Difficulté	Solution apportée
Absence de champ « secteur d'activité » dans l'API	Classification manuelle progressive par dictionnaire de correspondance domaine/secteur, en priorisant les breaches à fort volume
Nombreux breaches sans domaine identifiable (compilations)	Création d'une catégorie dédiée (« Compilation / Non attribuable ») plutôt qu'une classification artificielle
Création d'un compte AWS bloquée (absence de carte bancaire, échec répété de la vérification téléphonique depuis le Togo)	Utilisation du compte AWS d'un camarade de groupe, avec création d'un utilisateur IAM dédié à permissions limitées
Instabilité ponctuelle des conteneurs Docker (panne du moteur Docker Desktop, erreurs « read-only file system »)	Diagnostic via les journaux (logs) des conteneurs, redémarrage complet de Docker Desktop et de WSL2
Dépendance à S3 non disponible en cours de développement	Conception du DAG avec bascule automatique en mode simulation, permettant de développer et tester sans bloquer sur l'accès cloud
Fichiers sensibles (clés AWS) présents dans le dossier de travail local	Vérification systématique de l'historique Git avant chaque commit, exclusion stricte via .gitignore avant toute publication

9. Limites et esprit critique

- La classification par secteur d'activité reste partielle : une part significative des données demeure « Non classée », faute d'un champ dédié dans l'API source
- Les breaches de type « compilation » (stealer logs, combolists) n'ont pas de site source unique et ne peuvent, par nature, être rattachés à un secteur
- Le total cumulé de comptes compromis ne correspond pas à un nombre de personnes uniques, une même personne pouvant apparaître dans plusieurs breaches
- Les données de l'année 2026 sont incomplètes au moment de l'analyse, l'année n'étant pas terminée, et ne sont donc pas directement comparables aux années précédentes
- Le pipeline pourrait être enrichi par des alertes automatiques en cas d'échec d'exécution du DAG, non implémentées à ce stade

10. Récapitulatif des livrables attendus

Livrable	Statut
Dépôt GitHub public avec README, schéma d'architecture et docker-compose.yaml	Réalisé
Scripts Python d'ingestion (API), commentés	Réalisé
DAG Airflow fonctionnel avec au moins 3 tâches	Réalisé
Bucket S3 avec données organisées par date d'ingestion	Réalisé
Dashboard Power BI avec minimum 5 visualisations	Réalisé
Présentation orale avec démonstration du DAG en fonctionnement	Réalisé

11. Conclusion et perspectives

Ce projet a permis de construire un pipeline de données complet et fonctionnel, depuis la collecte automatisée des données jusqu'à leur visualisation interactive, en passant par une orchestration robuste et un stockage cloud structuré. Au-delà de l'aspect technique, ce travail a été l'occasion de résoudre des difficultés concrètes d'infrastructure (Docker, accès cloud) et de prendre des décisions méthodologiques justifiées, notamment concernant les limites de la classification par secteur d'activité, absente des données sources.

Pistes d'amélioration envisagées pour la suite du projet :

- Enrichir la classification par secteur d'activité via une base de correspondance domaine/secteur plus large
- Ajouter des alertes automatiques (email, Slack) en cas d'échec d'exécution du DAG
- Historiser et consolider plusieurs jours d'exécution sur S3, par exemple via DuckDB, pour des analyses de tendance plus fines